AGREGACIÓN, CONJUNTOS Y NULL

Clase 2 · Diseño de Base de Datos

Ingeniería en Informática · 2.º año · Universidad Católica de Córdoba

Agenda

0. **Recordatorio — qué venimos consultando**
1. **Funciones de agregación — `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`**
2. **`GROUP BY` — agrupar para resumir por categoría**
3. **`HAVING` — filtrar grupos · `WHERE` vs `HAVING` · orden de ejecución completo**
4. **Operaciones de conjuntos — `UNION`, `INTERSECT`, `EXCEPT`**
5. **`NULL` y lógica trivaluada — el tema que más errores silenciosos produce**

Al final vas a poder responder preguntas como "¿cuántos alumnos por carrera?" o "¿qué docentes nunca dictaron una comisión?".

De dónde venimos, a dónde vamos

CLASE 1 · CONSULTAS FILA POR FILA

Cada fila del resultado venía de una fila (o de un cruce de filas) de las tablas. `WHERE` elegía cuáles filas pasaban; `JOIN` combinaba filas de varias tablas.

CLASE 2 · RESUMIR Y COMBINAR CONJUNTOS

Ahora vamos a colapsar muchas filas en una sola (agregación), o a combinar resultados de varias consultas (conjuntos), o a manejar la ausencia de datos (`NULL`).

✓ ANTES DE SEGUIR

Asegurate de que la base `sigue` esté cargada en MySQL Workbench y que tengas a mano el DER de las 8 tablas núcleo.

01

Funciones de agregación

Colapsar muchas filas en un solo valor

Las cinco funciones de agregación

Función	Qué devuelve	Acepta
`COUNT(x)`	Cantidad de filas	Cualquier tipo (también `*`)
`SUM(x)`	Suma	Solo numérico
`AVG(x)`	Promedio (media)	Solo numérico
`MIN(x)`	Valor mínimo	Cualquier tipo ordenable (números, cadenas, fechas)
`MAX(x)`	Valor máximo	Cualquier tipo ordenable

```sql
-- Ejemplos sobre SiGUE
SELECT COUNT(*)             FROM alumnos;          -- total de alumnos
SELECT SUM(presupuesto)     FROM departamentos;    -- gasto total
SELECT AVG(nota)            FROM notas;            -- promedio general
SELECT MIN(fecha_ingreso)   FROM docentes;         -- docente más antiguo
SELECT MAX(salario_base)    FROM docentes;         -- sueldo más alto
```

Las tres caras de COUNT

Forma	Qué cuenta
`COUNT(*)`	Cantidad de **filas**. Cuenta cada fila, incluso si todas sus columnas son `NULL`.
`COUNT(columna)`	Cantidad de filas donde esa columna **no es** `NULL`.
`COUNT(DISTINCT columna)`	Cantidad de **valores distintos** (no nulos) en esa columna.

```sql
-- Sobre la tabla inscripcion_alumno_comision
SELECT COUNT(*)                FROM inscripcion_alumno_comision;  -- todas las inscripciones
SELECT COUNT(id_alumno)        FROM inscripcion_alumno_comision;  -- inscripciones con alumno cargado
SELECT COUNT(DISTINCT id_alumno) FROM inscripcion_alumno_comision; -- cuántos alumnos distintos se inscribieron
```

⚠ **OJO**

Si una columna admite `NULL`, `COUNT(*)` y `COUNT(columna)` pueden dar resultados **diferentes**. No son intercambiables.

Sin GROUP BY, todo es un único grupo

Una función de agregación sin `GROUP BY` reduce toda la tabla (filtrada por `WHERE`) a **una sola fila**.

```sql
-- Una fila de resultado: el promedio sobre TODAS las notas filtradas
SELECT AVG(nota) AS promedio_general
FROM notas
WHERE aprobado = 1;
```

⚠ MEZCLAR AGREGADO CON COLUMNA SUELTA = ERROR

```sql
-- INVÁLIDA: id_alumno es una columna y AVG(nota) es un único valor
-- ¿Qué id_alumno mostraría?
SELECT id_alumno, AVG(nota) FROM notas;
```

Para combinar columnas sueltas con agregados, hace falta `GROUP BY` — lo vemos ahora.

Cómo tratan al NULL las agregaciones

REGLA PRÁCTICA

Todas las funciones de agregación **ignoran los `NULL`**… salvo `COUNT(*)`, que cuenta filas sin importar su contenido.

Caso	Resultado
`AVG(salario_base)` con algunos sueldos nulos	Promedio sobre los **no nulos** (los nulos no cuentan ni como cero ni como filas).
`SUM(nota)` cuando todas las notas son `NULL`	Devuelve `NULL` (no `0`).
`COUNT(columna)` de un conjunto vacío	Devuelve `0`.
`AVG/SUM/MIN/MAX` de un conjunto vacío	Devuelve `NULL`.

Volvemos a NULL en la sección 5 — ahí se ve a fondo.

02

GROUP BY

Agrupar para resumir por categoría

GROUP BY: una fila de resultado por grupo

Arma grupos de filas que comparten el mismo valor en las columnas listadas, y aplica la agregación a cada grupo.

```sql
-- Cantidad de inscripciones por comisión
SELECT id_comision, COUNT(*) AS inscriptos
FROM inscripcion_alumno_comision
GROUP BY id_comision;
```

CÓMO SE LEE EL RESULTADO

El motor agrupa las inscripciones por `id_comision` (todas las de la comisión 1 juntas, las de la 2 juntas, etc.) y para cada grupo cuenta las filas. **Una fila de resultado por comisión.**

La regla de oro del GROUP BY

REGLA

Toda columna que aparezca en el `SELECT` **sin estar dentro de una función de agregación** debe estar también en el `GROUP BY`.

INVÁLIDA

```
SELECT id_asignatura,
       id_docente,
       COUNT(*)
FROM comisiones
GROUP BY id_asignatura;
```

id_docente está en SELECT pero no en GROUP BY ni dentro de un agregado.

VÁLIDA

```
SELECT id_asignatura,
       id_docente,
       COUNT(*)
FROM comisiones
GROUP BY id_asignatura, id_docente;
```

Ahora cada grupo es una combinación asignatura+docente.

⚠ OJO MYSQL

MySQL históricamente fue permisivo con esta regla y devolvía resultados arbitrarios. Desde 5.7 **la chequea por defecto** (`ONLY_FULL_GROUP_BY`). Si te aparece un error de "non-aggregated column", esto es lo que está pasando.

Agrupar por más de una columna

```sql
-- Cantidad de comisiones abiertas por año y cuatrimestre
SELECT anio,
       cuatrimestre,
       COUNT(*) AS cant_comisiones
FROM comisiones
GROUP BY anio, cuatrimestre
ORDER BY anio DESC, cuatrimestre;
```

Cada grupo es una combinación de `(anio, cuatrimestre)`. Si hay datos en 2024-1, 2024-2, 2025-1 y 2025-2, el resultado tiene 4 filas.

```sql
-- También funciona combinado con JOIN: promedio de notas por asignatura
SELECT asg.nombre, AVG(n.nota) AS promedio
FROM notas n
INNER JOIN examenes e   ON n.id_examen = e.id_examen
INNER JOIN comisiones c ON e.id_comision = c.id_comision
INNER JOIN asignaturas asg ON c.id_asignatura = asg.id_asignatura
GROUP BY asg.nombre;
```

03

HAVING y orden de ejecución

Filtrar grupos · WHERE vs HAVING

HAVING: filtrar grupos

`WHERE` filtra **filas** antes de agruparlas. `HAVING` filtra **grupos** después de armarlos. Por eso `HAVING` puede usar funciones de agregación; `WHERE` no.

```sql
-- Comisiones con más de 25 inscriptos
SELECT id_comision, COUNT(*) AS inscriptos
FROM inscripcion_alumno_comision
GROUP BY id_comision
HAVING COUNT(*) > 25;
```

⚠ ESTO NO COMPILA

```sql
SELECT id_comision, COUNT(*) FROM inscripcion_alumno_comision
WHERE COUNT(*) > 25      -- ✗ WHERE no admite agregados
GROUP BY id_comision;
```

Cuando `WHERE` corre, los grupos todavía no existen — no hay `COUNT(*)` que aplicar.

WHERE vs HAVING — resultados *distintos*

Misma pregunta aparente, semántica diferente: "Cantidad de exámenes por alumno"...

A · CON WHERE

```sql
SELECT id_alumno,
       COUNT(*) AS examenes
FROM notas
WHERE aprobado = 1
GROUP BY id_alumno;
```

Cuenta **solo las notas aprobadas**. Filtra filas antes de agrupar.

B · CON HAVING

```sql
SELECT id_alumno,
       COUNT(*) AS examenes
FROM notas
GROUP BY id_alumno
HAVING COUNT(*) > 5;
```

Cuenta **todas las notas** y descarta alumnos con 5 o menos. Filtra grupos.

LA PREGUNTA CORRECTA ANTES DE ELEGIR

¿Quiero descartar **filas individuales** antes de contar? → `WHERE`. ¿Quiero descartar **grupos enteros** según una métrica calculada? → `HAVING`.

Los tres juntos

```sql
-- Por asignatura, cuántas comisiones del ciclo 2025 tienen cupo > 30,
-- mostrando solo asignaturas con al menos 2 comisiones así.
SELECT id_asignatura, COUNT(*) AS comisiones_grandes
FROM comisiones
WHERE anio = 2025 AND cupo > 30
GROUP BY id_asignatura
HAVING COUNT(*) ⩾ 2
ORDER BY comisiones_grandes DESC;
```

LEÍDO PASO A PASO

- **WHERE**: descarta filas que no son del 2025 o tienen cupo ≤ 30.
- **GROUP BY**: agrupa las filas restantes por asignatura.
- **HAVING**: descarta grupos con menos de 2 comisiones.
- **ORDER BY**: ordena el resultado final por cantidad descendente.

Orden de ejecución, ahora completo

Lo escribís así:

`SELECT` → `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY`

El motor lo ejecuta así:

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`

CONSECUENCIAS PRÁCTICAS

- **Un alias del SELECT no se puede usar en WHERE**: cuando WHERE corre, SELECT todavía no se ejecutó.
- **Sí podés usar el alias en ORDER BY**: a esa altura SELECT ya proyectó.
- **HAVING puede usar agregados, WHERE no**: cuando WHERE corre, los grupos no existen.

Pausa: probalo vos

Consigna: listar los *departamentos* que tienen al menos 3 docentes, mostrando el nombre del departamento y la cantidad, ordenados de mayor a menor.

```sql
SELECT dep.nombre AS departamento,
       COUNT(*) AS cant_docentes
FROM docentes d
INNER JOIN departamentos dep
  ON d.id_departamento = dep.id_departamento
GROUP BY dep.nombre
HAVING COUNT(*) ⩾ 3
ORDER BY cant_docentes DESC;
```

VARIANTE

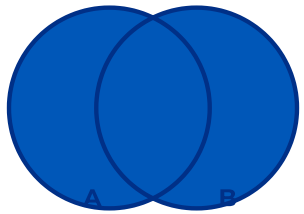
¿Por qué `HAVING COUNT(*) ⩾ 3` y no `WHERE COUNT(*) ⩾ 3`? Cuando `WHERE` corre, los grupos aún no existen — no hay nada que contar.

04

Operaciones de conjuntos

Combinar resultados de varias consultas

UNION: A ∪ B




```sql
-- Emails de todos los miembros activos de la universidad
SELECT email FROM alumnos
UNION
SELECT email FROM docentes;
```

Combina las filas de las dos consultas en un único resultado. Por defecto **elimina duplicados**.

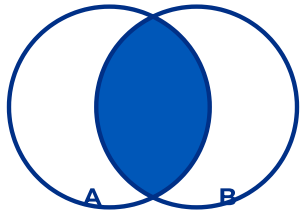
UNION VS UNION ALL

`UNION` quita duplicados (costoso). `UNION ALL` los conserva (más rápido). Si sabés que no puede haber duplicados, usá `UNION ALL`.

⚠ REQUISITO

Las dos consultas deben tener la **misma cantidad de columnas** y tipos compatibles. Los nombres de columna los pone la primera consulta.

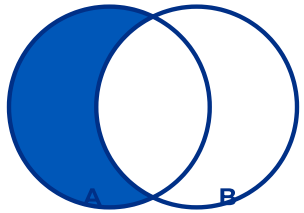
INTERSECT: A ∩ B




```sql
-- Asignaturas dictadas en 2024-1 Y en 2024-2
SELECT id_asignatura FROM comisiones
WHERE anio = 2024 AND cuatrimestre = 1
INTERSECT
SELECT id_asignatura FROM comisiones
WHERE anio = 2024 AND cuatrimestre = 2;
```

Solo las filas que **aparecen en ambas** consultas. También elimina duplicados.

EXCEPT: A − B




```sql
-- Asignaturas obligatorias que NO tienen ninguna comisión abierta en 2025
SELECT id_asignatura FROM asignaturas
WHERE es_obligatoria = 1
EXCEPT
SELECT DISTINCT id_asignatura FROM comisiones
WHERE anio = 2025;
```

Filas que están en **A pero no en B**.

✓ **TIP**

En Oracle se llama `MINUS` en lugar de `EXCEPT`. El comportamiento es el mismo.

MySQL: INTERSECT y EXCEPT son recientes

> ⚠ **VERSIÓN**
>
> MySQL **recién soporta `INTERSECT` y `EXCEPT` desde la versión 8.0.31** (octubre de 2022). En versiones anteriores, hay que reescribir usando `IN`, `NOT IN` o `EXISTS`.

`INTERSECT` reescrito con `IN`:

```sql
-- Asignaturas dictadas en ambos cuatrimestres de 2024
SELECT DISTINCT id_asignatura FROM comisiones
WHERE anio = 2024 AND cuatrimestre = 1
  AND id_asignatura IN (
    SELECT id_asignatura FROM comisiones
    WHERE anio = 2024 AND cuatrimestre = 2
  );
```

`EXCEPT` reescrito con `NOT IN`:

```sql
-- Asignaturas obligatorias sin comisiones en 2025
SELECT id_asignatura FROM asignaturas
WHERE es_obligatoria = 1
  AND id_asignatura NOT IN (
    SELECT id_asignatura FROM comisiones WHERE anio = 2025
  );
```

05

NULL y lógica trivaluada

El tema que más errores silenciosos produce

NULL no es cero, no es vacío, no es nada

DEFINICIÓN

`NULL` representa la **ausencia de información**: "no sé qué valor tiene este atributo en esta fila". Es distinto de cero, distinto del string vacío, distinto de "ninguno".

Caso real en SiGUE	Qué significa el NULL
`alumnos.telefono` = `NULL`	No se sabe el teléfono. Puede que tenga, puede que no.
`notas.nota` = `NULL`	El examen está registrado pero la nota todavía no se cargó.
`docentes.id_departamento` = `NULL`	El docente todavía no fue asignado a un departamento.

⚠ ANTES DE SEGUIR

El comportamiento de `NULL` es **distinto al de cualquier otro valor**. Lo que vas a ver en los próximos slides explica por qué muchas consultas "raras" en realidad están funcionando correctamente.

Aritmética con NULL: contagia todo

REGLA

Cualquier operación aritmética donde participe un `NULL` devuelve `NULL`.

```sql
-- Si salario_base es NULL para alguna fila, el resultado es NULL en esa fila
SELECT apellido, salario_base + 5000 AS con_bono
FROM docentes;

-- También con multiplicación, división, concatenación
-- NULL * 1.10 = NULL
-- NULL || 'x' = NULL (en MySQL: CONCAT(NULL, 'x') depende del modo)
```

✓ CÓMO "NEUTRALIZAR" UN NULL

MySQL ofrece `IFNULL(col, valor_si_null)` y el estándar `COALESCE(col1, col2, ...)` devuelve el primero no nulo.

```sql
SELECT apellido, IFNULL(salario_base, 0) + 5000 FROM docentes;
```

Lógica trivaluada: TRUE, FALSE, UNKNOWN

Las comparaciones contra `NULL` no son ni verdaderas ni falsas: son **UNKNOWN**.

```sql
-- Todas estas comparaciones devuelven UNKNOWN, no TRUE ni FALSE
NULL = 5            -- UNKNOWN
NULL = NULL         -- UNKNOWN  (no es TRUE)
NULL <> NULL        -- UNKNOWN
NULL > 0            -- UNKNOWN
```

REGLA DEL WHERE

Una fila entra al resultado **solo si su predicado evalúa a TRUE**. Las filas con `UNKNOWN` se descartan, igual que las `FALSE`.

```sql
-- Esto NO devuelve los docentes con id_departamento desconocido
SELECT * FROM docentes WHERE id_departamento = NULL;
-- Resultado: 0 filas. El predicado es UNKNOWN para todas.
```

Tablas de verdad con UNKNOWN

AND

	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

OR

	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

NOT

x	NOT x
T	F
F	T
U	U

CASOS INTERESANTES

- `FALSE AND UNKNOWN = FALSE` — si una parte ya es falsa, el todo lo es.
- `TRUE OR UNKNOWN = TRUE` — si una parte ya es verdadera, el todo lo es.
- `NOT UNKNOWN = UNKNOWN` — "no sé" sigue siendo "no sé".

La forma correcta: IS NULL / IS NOT NULL

Para preguntar si un atributo es `NULL`, hay que usar `IS NULL` (no `= NULL`).

```sql
-- Docentes que todavía no fueron asignados a un departamento
SELECT apellido, nombre
FROM docentes
WHERE id_departamento IS NULL;

-- Alumnos con teléfono cargado
SELECT apellido, telefono
FROM alumnos
WHERE telefono IS NOT NULL;
```

✓ **MYSQL ESPECÍFICO**

MySQL acepta también el operador ⟺ ("null-safe equal"): `a ⟺ b` devuelve TRUE si ambos son NULL, FALSE si solo uno es NULL. No está en el estándar — uso restringido.

NULL en agregaciones y DISTINCT

EN AGREGACIONES

Las funciones `SUM`, `AVG`, `MIN`, `MAX` y `COUNT(col)` **ignoran** los NULL: actúan como si esas filas no existieran. Solo `COUNT(*)` los cuenta.

EN DISTINCT Y GROUP BY

Acá pasa lo contrario: **dos `NULL` se consideran iguales**. Todos los NULL forman un solo grupo.

```sql
-- Si hay 100 alumnos y 30 tienen telefono = NULL
SELECT COUNT(*)        FROM alumnos;       -- 100
SELECT COUNT(telefono) FROM alumnos;       -- 70

-- DISTINCT considera todos los NULL como uno solo
SELECT COUNT(DISTINCT id_departamento) FROM docentes;
-- Si hay docentes con id_departamento NULL, NO se cuenta (COUNT ignora NULL).
```

⚠ INCONSISTENCIA APARENTE

El mismo `NULL` se trata distinto según el contexto: **en `WHERE` da UNKNOWN**, **en `GROUP BY` forma un grupo propio**, **en `COUNT(col)` se ignora**. No es un bug, es el estándar SQL.

Pausa: el NULL traicionero

Pregunta: "alumnos que *no* están inscriptos en la comisión 7". Alguien escribe:

```sql
SELECT apellido, nombre
FROM alumnos
```

⚠ SI LA SUBCONSULTA DEVUELVE ALGÚN NULL, ESTE SELECT DA 0 FILAS

Si en `inscripcion_alumno_comision` hay alguna fila con `id_alumno = NULL`, el resultado completo se vacía. ¿Por qué? Porque `X NOT IN (a, b, NULL)` se evalúa como `X <> a AND X <> b AND X <> NULL`, y esa última comparación devuelve UNKNOWN → la fila no entra.

✓ SOLUCIÓN

```sql
-- Opción A: filtrar NULL en la subconsulta
WHERE id_alumno NOT IN (
  SELECT id_alumno FROM inscripcion_alumno_comision
  WHERE id_comision = 7 AND id_alumno IS NOT NULL
)

-- Opción B (mejor): usar NOT EXISTS, que es inmune a NULL
WHERE NOT EXISTS (...)
```

`EXISTS` y `NOT EXISTS` son tema de la próxima clase.

Lo que vimos hoy

- **5 funciones de agregación**: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
- **`GROUP BY`**: arma grupos · regla de oro de columnas.
- **`HAVING`** filtra grupos · `WHERE` filtra filas.
- **Orden de ejecución**: FROM → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
- **Conjuntos**: `UNION`/`UNION ALL`, `INTERSECT`, `EXCEPT`.
- En MySQL `INTERSECT` y `EXCEPT` son desde 8.0.31; antes, reescribir con `IN`/`NOT IN`.
- **NULL** es ausencia de información, no cero.
- **Lógica trivaluada**: TRUE, FALSE, UNKNOWN. `IS NULL` en lugar de `= NULL`.

PRÓXIMA CLASE · SUBCONSULTAS, VISTAS Y MODIFICACIÓN

Subconsultas (`IN`, `EXISTS`, escalares, en `FROM`, `WITH`), vistas (`CREATE VIEW`) y modificación con `INSERT` / `UPDATE` / `DELETE`.

Práctico de la semana: consultas con agregación + grupos + NULL sobre SiGUE.